МИЭМ

Москва
2026

Разработка и эксплуатация защищенных систем искусственного интеллекта (MLSecOps)

Рыбаков Пётр Владимирович

- Выпускник МИЭМ
- Преподаю в МИЭМ компьютерное зрение и программирование
- Работаю в Центре искусственного интеллекта ВШЭ



Цели на сегодня:

- Рассмотреть общую постановку задачи машинного обучения
- Познакомиться с основными терминами МО
- Посмотреть на практике на работу простых алгоритмов: линейной регрессии и классификатора kNN

Почему это важно для MLSecOps

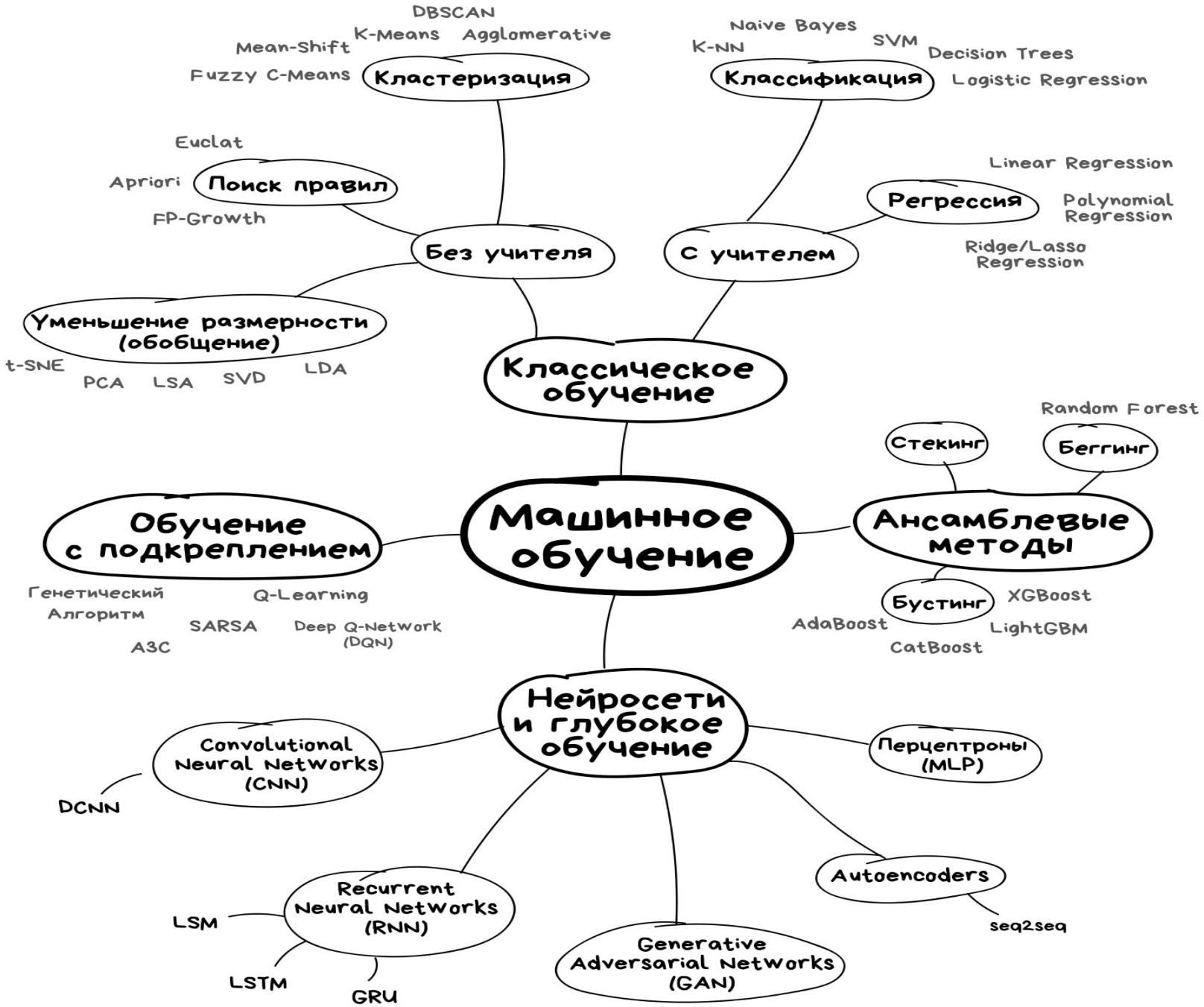
Зачем это может быть нужно вам?

При защите ML-систем важно понимать, как строятся ML-модели и на каких этапах этого строительства могут возникать риски.

Почему это важно для MLSecOps

- качество модели зависит от данных;
- ошибки модели могут быть эксплуатационными рисками;
- важно контролировать этапы обучения и оценки;
- нужна воспроизводимость экспериментов;
- метрики — способ понять, можно ли доверять модели.

Методы


Машинное обучение
Классическое обучение
Без учителя
Кластеризация
DBSCAN
K-Means
Agglomerative
Mean-Shift
Fuzzy C-Means
Поиск правил
Euclat
Apriori
FP-Growth
Уменьшение размерности (обобщение)
t-SNE
PCA
LSA
SVD
LDA
С учителем
Классификация
Naive Bayes
K-NN
SVM
Decision Trees
Logistic Regression
Регрессия
Linear Regression
Polynomial Regression
Ridge/Lasso Regression
Обучение с подкреплением
Генетический Алгоритм
Q-Learning
SARSA
Deep Q-Network (DQN)
A3C
Ансамблевые методы
Стекинг
Беггинг
Random Forest
Бустинг
XGBoost
AdaBoost
LightGBM
CatBoost
Нейросети и глубокое обучение
Convolutional Neural Networks (CNN)
DCNN
Перцептроны (MLP)
Recurrent Neural Networks (RNN)
LSM
LSTM
GRU
Generative Adversarial Networks (GAN)
Autoencoders
seq2seq

Классическое Обучение

Данные заранее категоризированы или численные

Данные никак не размечены

С учителем

Предсказать категорию

Классификация

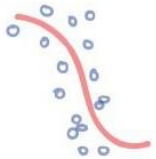
«Разложи носки по цвету»



Предсказать значение

Регрессия

«Разложи галстуки по длине»

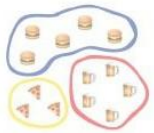


Без учителя

Разделить по схожести

Кластеризация

«Разложи похожие вещи по кучкам»



Найти зависимости

Уменьшение Размерности (обобщение)

«Собери из вещей лучшие наряды»

Выявить последовательности

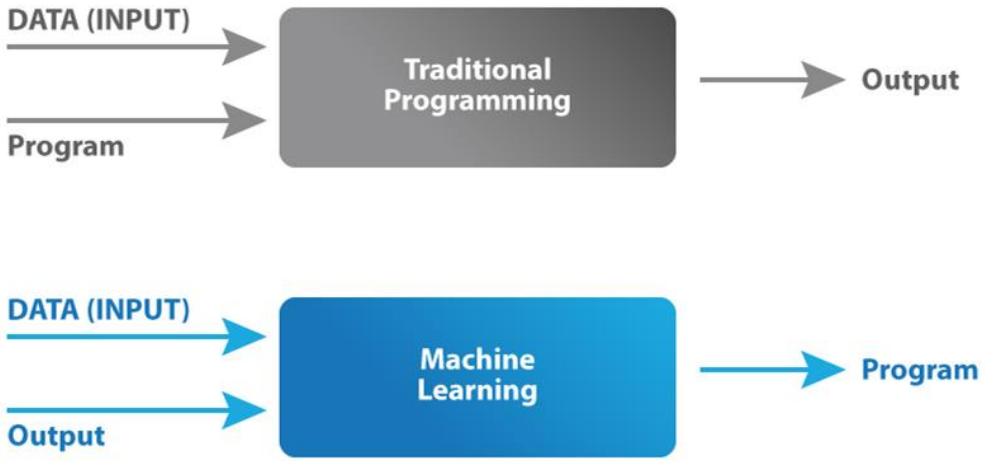
Ассоциация

«Найди какие вещи я часто ношу вместе»



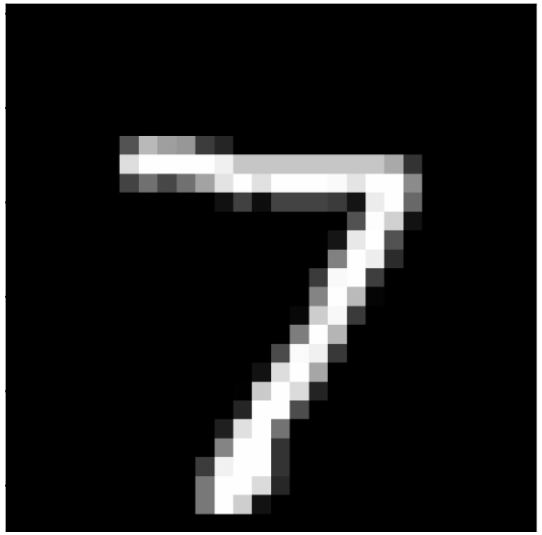
Что за ML?

- ML — это парадигма аппроксимации функции на основе данных
  - Функция здесь представляет собой просто набор правил, который принимает входные данные и выдает некоторые выходные данные (например, метку или прогнозируемое значение)



Почему ML, а не самостоятельно запрограммированные функции?

- Иногда мы не можем понять закономерности в наших данных, поэтому очень сложно придумать эти правила!
- ML — это, по сути, процесс, при котором наши данные определяют создание функции.



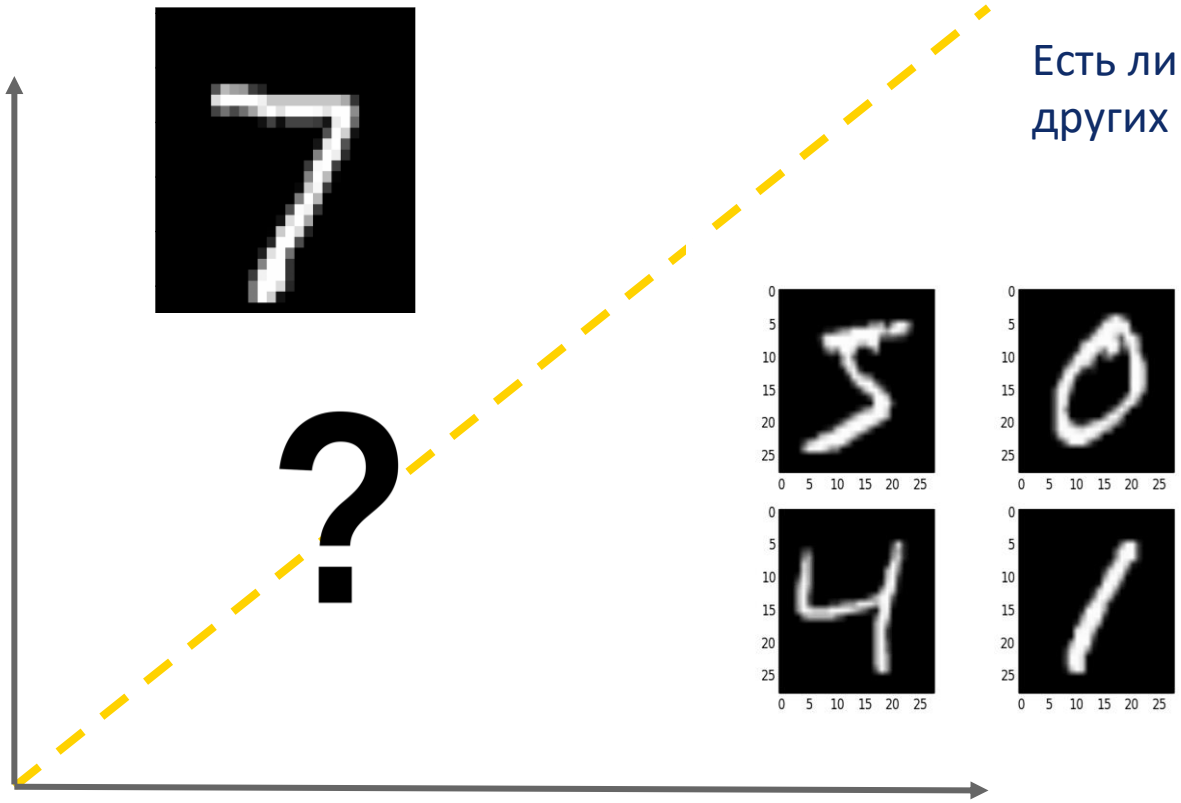
input

```
1 def classify_image(image: list[list[int]]) -> int:
2     # TODO: Classify the image to return 7!
3     # ... how do we proceed?
4     return
```

Nope

```
for i in range(10, 30):
            if image[10][i] > 0.5:
                        count += 1
if low_thresh < count < high_thresh:
            return 7
```

Задачка: попробуйте написать функцию для классификации чисел?



Есть ли способ отделить 7 от других цифр?

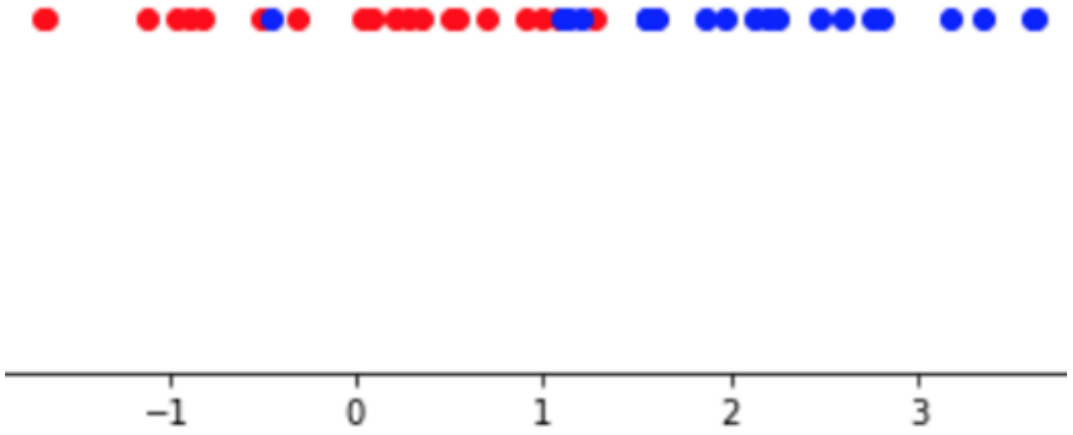
Задачка: попробуйте написать функцию для классификации чисел?

Приближаемся к сути ML

- Представьте себе, что это создание шаблона
  - Когда мы обычно определяем функцию вручную, нам приходится указывать ВСЁ.
  - С помощью ML мы определим функцию (с помощью математики), но оставим несколько свободных параметров, которые будут обучаться на данных: они будут диктовать точное поведение функции.
  - Пока что не задумывайтесь о процессе обучения правильному выбору параметров... это будет позже!
- Пример:
- Мы определим нашу функцию в следующем виде: if (input < a) –> output1, else –> output2,и узнаем лучшее значение a из наших данных.
- Здесь «a» — свободный параметр, который определяет точное поведение нашей функции-примера.

```
def function(input, a):
    """
    input (int) - our function's input
    a (int) - our parameter that we will learn

    Returns:
    True / False classification
    """
    if input < a:
        return 'red'
    else:
        return 'blue'
```

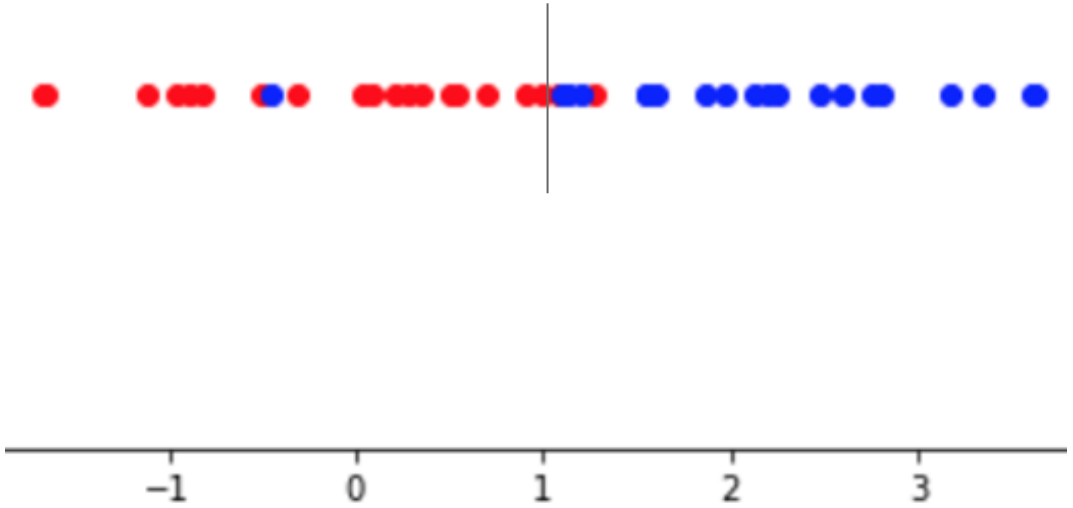



```python
def function(input, a):
    """
    input (int) - our function's input
    a (int) - our parameter that we will learn

    Returns:
    True / False classification
    """
    if input < a:
        return 'red'
    else:
        return 'blue'
```

Примечание: эта функция — всего лишь предполагаемая функция, которая, как мы надеемся, будет работать хорошо, исходя из того, как выглядят данные.

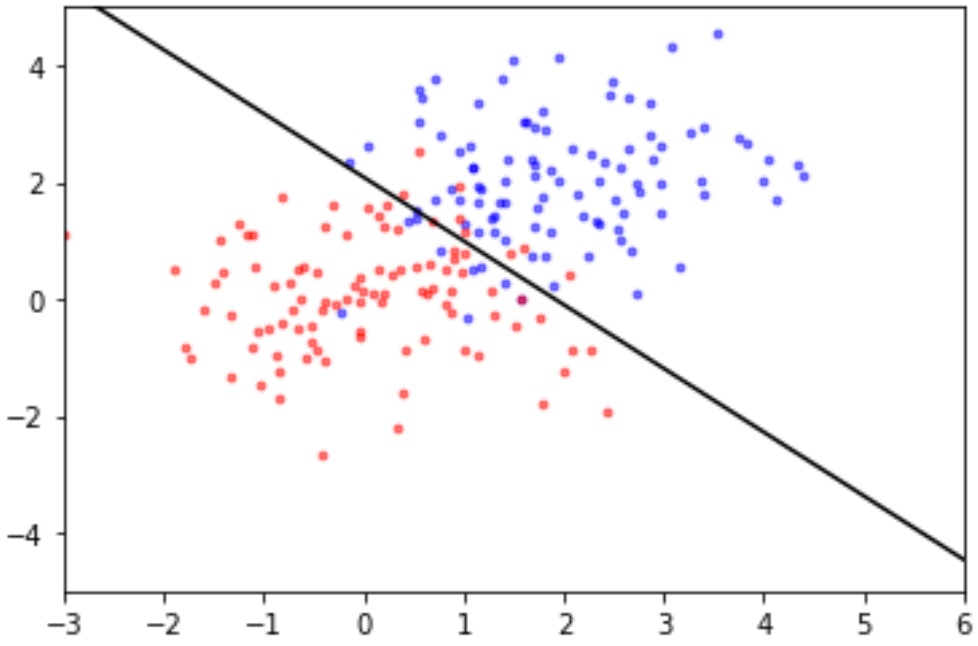
Какое значение ‘a’ будет хорошим?




```python
def function(input, a):
    """
    input (int) - our function's input
    a (int) - our parameter that we will learn

    Returns:
    True / False classification
    """
    if input < a:
        return 'red'
    else:
        return 'blue'
```

Какое значение ‘a’ будет хорошим? Наверное, a = 1



Раньше у нас была одна точка, выше которой всё было синим, а в остальных случаях — красным. Однако эта стратегия не работает в двумерном пространстве…

Теперь мы можем предположить, что данные разделяет одномерная линия, выше которой все точки синие, а ниже — красные.

Это наша ФУНКЦИЯ, существование которой мы предполагаем… одномерной линии вида

$y = mx+b$

В данном случае нашими параметрами являются m (наклон) и b (точка пересечения/смещение).

Двухмерный пример

Приближаемся к сути ML

Машинное обучение нужно там, где:

- много данных;
- правила сложны или неформализуемы;
- нужно обобщать на новые примеры.

Тип задачи	Пример
Регрессия	предсказание цены, спроса, времени
Классификация	спам / не спам, тип объекта, нормальное / аномальное поведение
Кластеризация	группировка клиентов, поиск похожих событий

Приближаемся к сути ML

- Искусство машинного обучения заключается в следующем:
  - Какую форму принимает наша функция → это можно назвать классом модели
  - Какие конкретные части этой функции нам разрешено изучать → это наши параметры
  - Как мы изучаем эти параметры, чтобы приблизить их к «наилучшим» возможным значениям
    - Мы поговорим об этом подробнее позже
- Каждый алгоритм машинного обучения, который вы когда-либо изучите, следует этому шаблону
  - Опишите общую форму функции со свободными параметрами
  - Используйте данные, чтобы решить, какие свободные параметры будут работать лучше всего
- Это крайне важно, **ПОЖАЛУЙСТА, ЗАДАВАЙТЕ ВОПРОСЫ, ЕСЛИ ОНИ У ВАС ЕСТЬ, ПОЖАЛУЙСТА, ЗАДАВАЙТЕ ИХ**

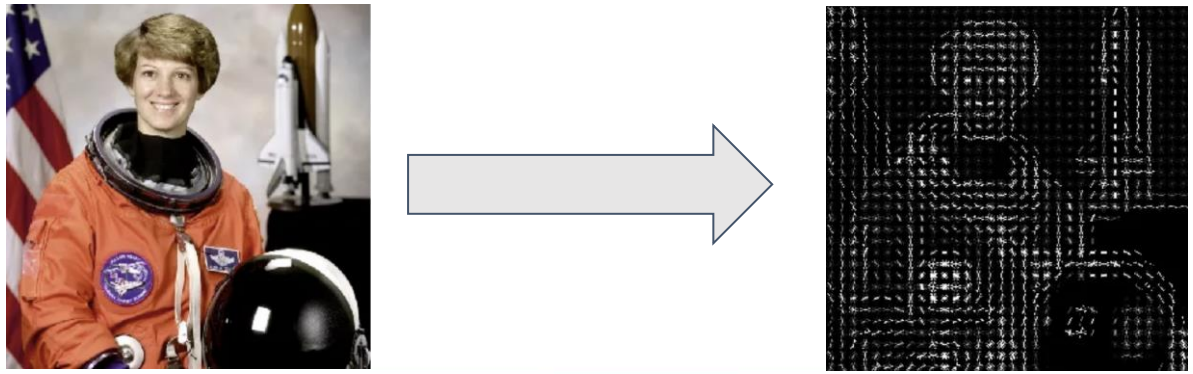
Термины

- Функция/Модель
  - Эти термины взаимозаменяемы.
  - Они относятся к шаблону функции («классу модели»), который мы выбрали для нашей задачи.
- Веса (и смещения)
  - Другой способ обозначения параметров в моделях машинного обучения, которые обучаются на основе данных.
- Гиперпараметр
  - Это некоторый необучаемый параметр (например, размер модели, тип модели, сведения о процедуре обучения и т. д.), который дополнительно определяет нашу общую обучаемую функцию.
  - Нам необходимо вручную выбрать их самостоятельно, прежде чем мы начнём обучать обучаемые параметры.
- Функция потерь/Функция стоимости/Функция риска
  - Мы ещё не вводили эти термины, но они будут рассмотрены позже; просто отметим, что они одинаковы (по крайней мере, для наших целей).

Термины

«Признак»

- Это может относиться к фрагментам наших данных (как к самим входным данным, так и к их представлению), которые мы передаем в качестве входных данных модели.
- Например: для входных данных изображения вы сжимаете его значения пикселей в вектор ИЛИ извлекаете из него такие элементы, как углы, края и формы — это разные «характеристики» одного и того же изображения, которые можно передать в модель!

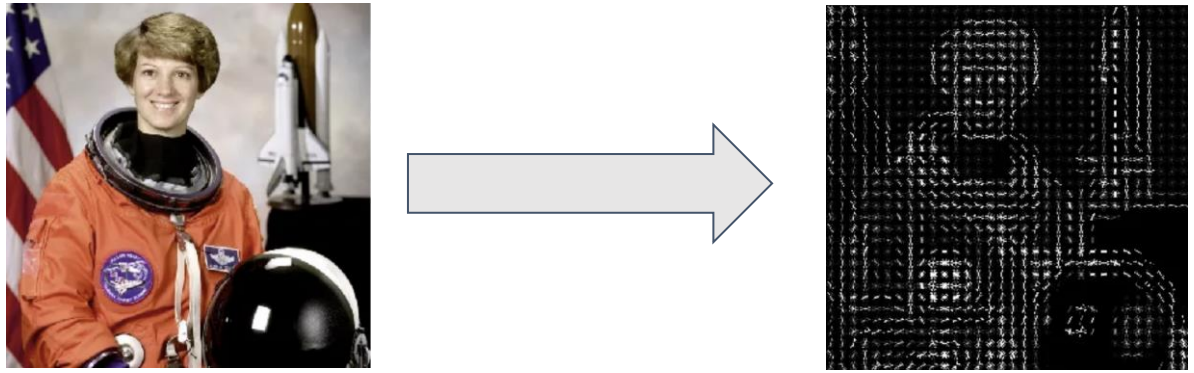


Термины

«Признак»

Примеры для регрессии цен квартир:

- площадь квартиры;
- число комнат;
- этаж;
- район.



Термины

«Признак»

Примеры для классификации цветов:

- длина лепестка;
- ширина лепестка;
- длина чашелистика;
- ширина чашелистика.







Определить задачу

1. Определить задачу
2. Подготовить данные
3. Определить модель и функцию потерь
4. Минимизировать функцию потерь (обучить модель)
5. ГОТОВО!

Определить задачу

ML пайплайн

1. Какую задачу вы пытаетесь решить с помощью машинного обучения?
2. Каковы ваши входные данные?
3. Какими должны быть ваши выходные данные?
4. Какова наша метрика успеха на уровне проекта? Чего мы надеемся достичь?

Подготовить данные

Определение задачи

Что должна нам дать наша модель в итоге?

- Классификация (ирисы Фишера):
  - класс ириса = f(длина лепестка, ширина лепестка, ...)
- Регрессия (цены на квартиры):
  - цена = f(площадь, число комнат, этаж, ...)

Типичные ошибки

Что должна нам дать наша модель в итоге?

- неверно выбрана целевая переменная;
- метрика не соответствует бизнес-задаче;
- утечка целевой переменной или признаков из будущего;
- оценка модели на обучающей выборке;
- игнорирование дисбаланса классов;
- сравнение моделей по одной случайной метрике без валидации.

Подготовка данных

- Сбор данных
  - Не принимайте это как должное в реальном мире… мусор на входе ⇒ мусор на выходе
- Нам нужно представить наши данные в виде чисел
  - Нам нужно перейти от текста к числам
  - Нам нужно перейти от файлов изображений к числам
  - Каждая точка данных должна быть каким-то образом представлена в виде чисел
- Выбор/масштабирование признаков
  - Определение того, какие части данных важны и должны быть включены в качестве входных данных для модели
  - Возможно, потребуется изменить масштаб некоторых признаков, чтобы все они находились в одном диапазоне значений: нормализация
- Векторы — одно из самых простых и важных представлений данных
  - Проще говоря, берём важные числа и помещаем их все в вектор (одномерную матрицу) в определённом порядке

Подготовка данных

Возможные проблемы:

- Данных недостаточно
- В данных есть пропуски
- В данных есть странные значения (выбросы)
- В данных есть текстовые/категориальные признаки
- Признаки совсем не похожи друг на друга
- Утечка данных

Подготовка данных

Возможные проблемы:

- Данных недостаточно - аугментация
- В данных есть пропуски – заполнение средним/медианой/синтетикой
- В данных есть странные значения (выбросы) – отброс/обработка как пропусков
- В данных есть текстовые/категориальные признаки – кодирование признаков
- Признаки совсем не похожи друг на друга - масштабирование
- Утечка данных – перепроверка чистоты данных

Пример: представление меток

One Hot кодирование

- Одна из наиболее распространённых схем маркировки для многоклассовой классификации
  - Классификация — это задача, в которой модель должна различать n различных типов входных данных, например, задача распознавания цифр.
- Вместо метки «4» для обозначения 4-го класса, задайте следующий вид:
  - [0, 0, 0, 1, 0, 0, 0, …]
- Другими словами, поместите 1 на i-ю позицию вектора, состоящего из нулей, чтобы обозначить i-й класс.
- Эта схема позволяет рассматривать метки как распределения вероятностей.
  - Вместо того, чтобы просто сказать, что точка данных помечена как класс 4 (см. пример выше), мы можем сказать, что она имеет 100% вероятность принадлежности к классу 4 и 0% вероятность принадлежности к любому другому классу.
  - Это особенно полезно, поскольку, как мы увидим в следующий раз, наши модели также будут выводить распределение вероятностей по классам. Например, [0, 0, 0,1, 0,75, 0,15, 0, 0, …] может быть выходом, когда модель полагает, что образец имеет 10% вероятность принадлежности к классу 3, 75% вероятность для класса 4 и 15% для класса 5.

Аугментация данных

- Нам может понадобиться больше данных, чем у нас есть. Что мы можем сделать?
- Мы найдём несколько преобразований, которые семантически не изменяют наши данные, то есть и входные данные, и их преобразованная версия должны иметь одинаковую метку.
- Изображения:
  - Мы можем добавить шум к изображениям или слегка размыть/повысить их резкость.
  - Мы можем повернуть изображения или немного деформировать их.
- Текст:
  - Мы можем заменить некоторые слова известными синонимами.
- Это искусственно даёт нам больше примеров для использования во время обучения.

Аугментированные данные





Примечание: прежде чем выбирать аугментацию, внимательно изучите, как выглядят ваши данные. Повёрнутая 9 всё ещё остаётся 9?

Разделение данных

В машинном обучении (ML) мы хотим знать, насколько хорошо наши системы обобщают данные. Мы хотим увидеть, насколько хорошо эти модели работают на данных, с которыми они ранее не сталкивались.

- ML бесполезно, если оно не может работать с новыми данными в реальном мире.
- Нам нужны конкретные данные, которые мы отложим для проверки обобщения: данные, которые наша модель ранее не встречала на этапе обучения.

Мы разделяем наши данные на три части (соотношение этих частей варьируется):

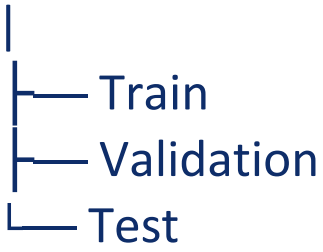
- Обучающие данные: данные, используемые для оптимизации параметров.
- Проверочные данные: данные, используемые для диагностики этапа обучения; чтобы помочь выбрать типы моделей и методов, которые лучше всего подходят для решения текущей задачи.
- Тестовые данные: данные, используемые для проверки обобщения модели только в самом конце процесса.

Существуют более продвинутые способы сделать это (здесь они не рассматриваются).

- Например K-fold кросс-валидация

Разделение данных

Все данные



Определить модель и функцию потерь

Определить модель

- Здесь мы определяем нашу модель.
  - В этом курсе мы будем говорить о различных типах моделей; не волнуйтесь, если вы пока не знакомы ни с одной из них.
- Какую модель можно использовать для решения нашей задачи МО?
  - Некоторые методы моделирования работают только для очень узких задач.
- Какая модель лучше всего отражает структуру наших данных?
  - Возможно, вы не сразу это поймете, но пока просто попытайтесь сделать обоснованное предположение!
  - Запомните, как должны выглядеть выходные данные вашей модели:
    - Будет ли это одно значение, несколько значений, изображения, текст и т. д.?
  - Мы также можем обучать разные модели на наших обучающих данных, тестировать их на проверочных и выбирать наиболее эффективную.
    - Это называется настройкой гиперпараметров.
  - Не перегружайте тестами модели, качество важнее количества.

Определить модель

Примеры:

- Классификация – kNN/логистическая регрессия
- Регрессия – линейная регрессия/градиентный бустинг
  - При должной сноровке под регрессию переделывается почти любая модель классификации
- Гиперпараметры:
- скорость обучения;
- число соседей k в KNN;
- сила регуляризации;
- максимальная глубина дерева;
- число итераций.

Определить функцию ошибки

- Напоминание: мы хотим обучить оптимальный набор параметров.
  - Нам нужна метрика для оптимизации.
  - Разные модели используют разные способы обучения конкретных параметров, но ВСЕ они будут пытаться оптимизировать какую-то метрику/функцию.
- После того, как у вас есть модель и данные, ваша задача — минимизировать функцию потерь.
  - Высокие потери ⇒ плохие параметры, низкие потери ⇒ хорошие параметры.
- Пример: обучение с учителем.
  - В обучении с учителем мы хотим, чтобы выходные данные нашей модели соответствовали некоторым меткам, которые являются векторами одинаковой формы.
  - Мы можем определить потерю как среднеквадратичную ошибку между нашими метками и прогнозами модели.

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Резюме

Термин	Кратко	Пример
Признак	входная характеристика	площадь дома
Модель	обучаемая функция	предсказание цены
Веса	параметры, которые обучаются	коэффициенты регрессии
Гиперпараметры	параметры, задаются заранее	k в KNN
Функция потерь	мера ошибки модели	MSE

Обучите модель, минимизируйте ошибку

Обучение

- К настоящему моменту мы выбрали данные, модель и функцию потерь.
- Сейчас мы применяем различные алгоритмы для выбора наилучших параметров на основе обучающего набора данных.
- Разные модели имеют разные процедуры обучения.
  - Мы рассмотрим их при описании каждой модели.

Готово!

Готово! (нет)

Готово! (нет)

мы забыли оценить результат

Готово! (нет)

мы забыли оценить результат

Метрики регрессии

Метрика	Что означает	Когда полезна
MAE	средняя абсолютная ошибка	когда важна интерпретируемость
MSE	средний квадрат ошибки	когда большие ошибки особенно плохи
RMSE	ошибка в единицах цели	удобна для интерпретации
R^2	качество относительно среднего	полезно для сравнения с наивной моделью

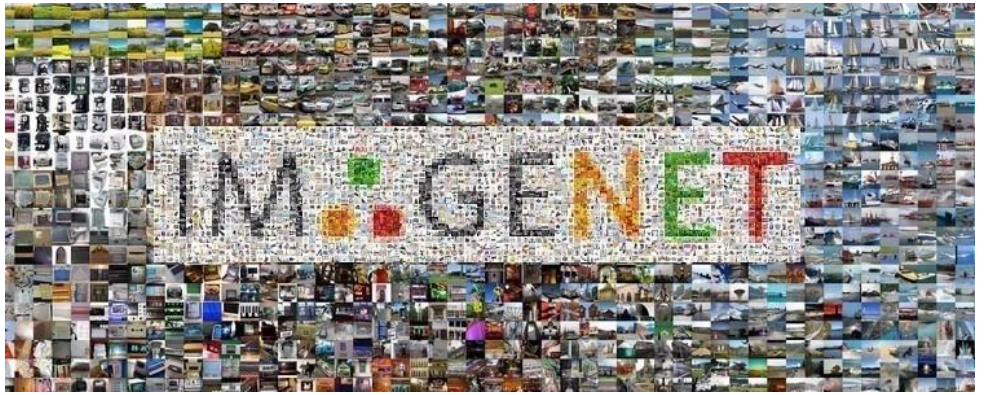
Метрики классификации

Accuracy	общая доля верных ответов	сбалансированные классы
Precision	насколько можно доверять положительным прогнозам	высокая цена ложных тревог
Recall	способность найти все положительные случаи	высокая цена пропущенных случаев
F1	баланс precision и recall	нужен компромисс
Log loss	уверенность вероятностей	важна калибровка вероятностей
Accuracy	общая доля верных ответов	сбалансированные классы

Завершающие шаги

- Используйте свой тестовый набор для оценки итоговой эффективности вашей модели.
- Если это распространённый, широкодоступный набор данных, вы можете использовать его для сравнения своих результатов с результатами современных систем и сравнить свои результаты с ними.
  - MNIST (это набор данных рукописных цифр, который вы видели ранее) часто используется для проверки концепций.
  - Imagenet — важный эталон для классификации и генерации изображений.

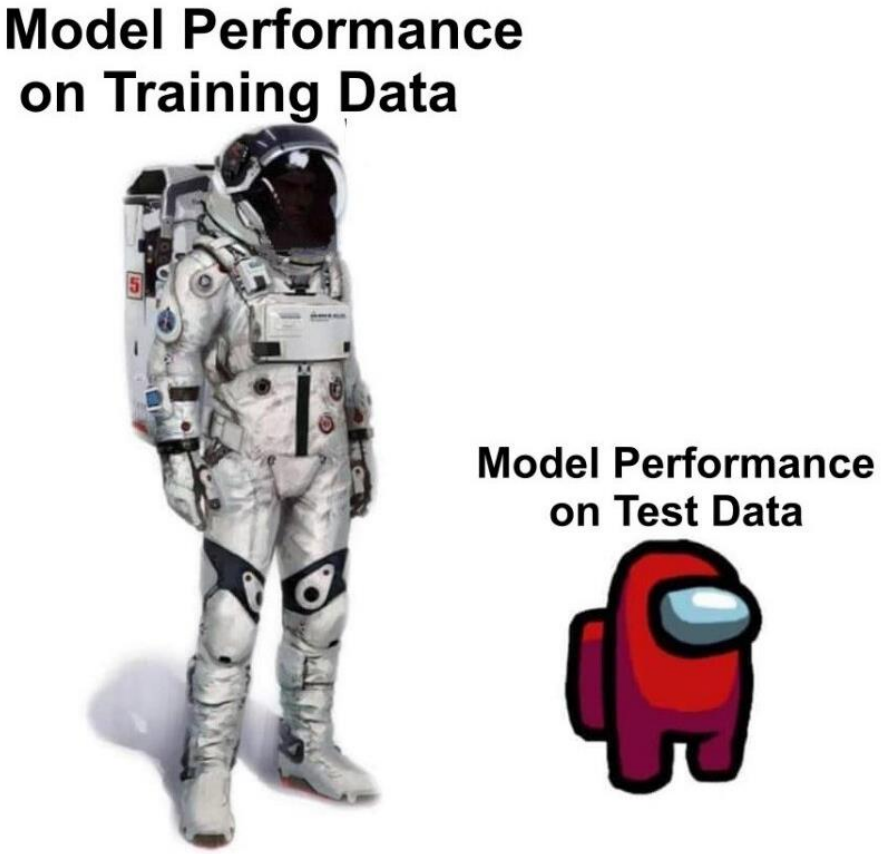




Обобщение в ML

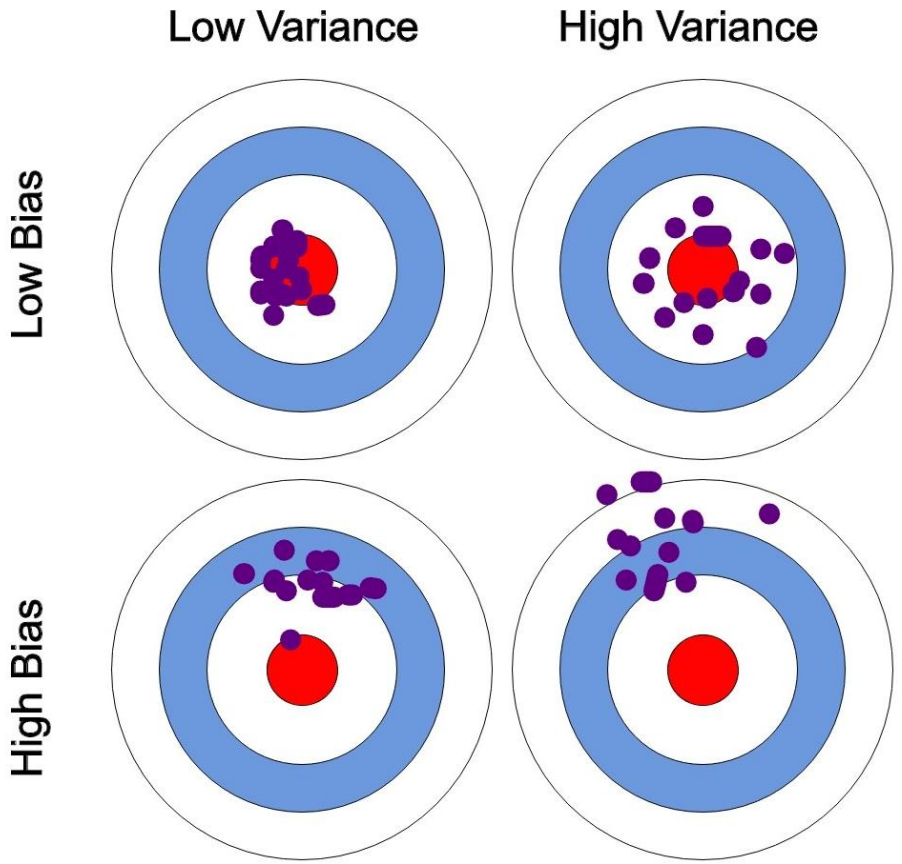
Обобщение

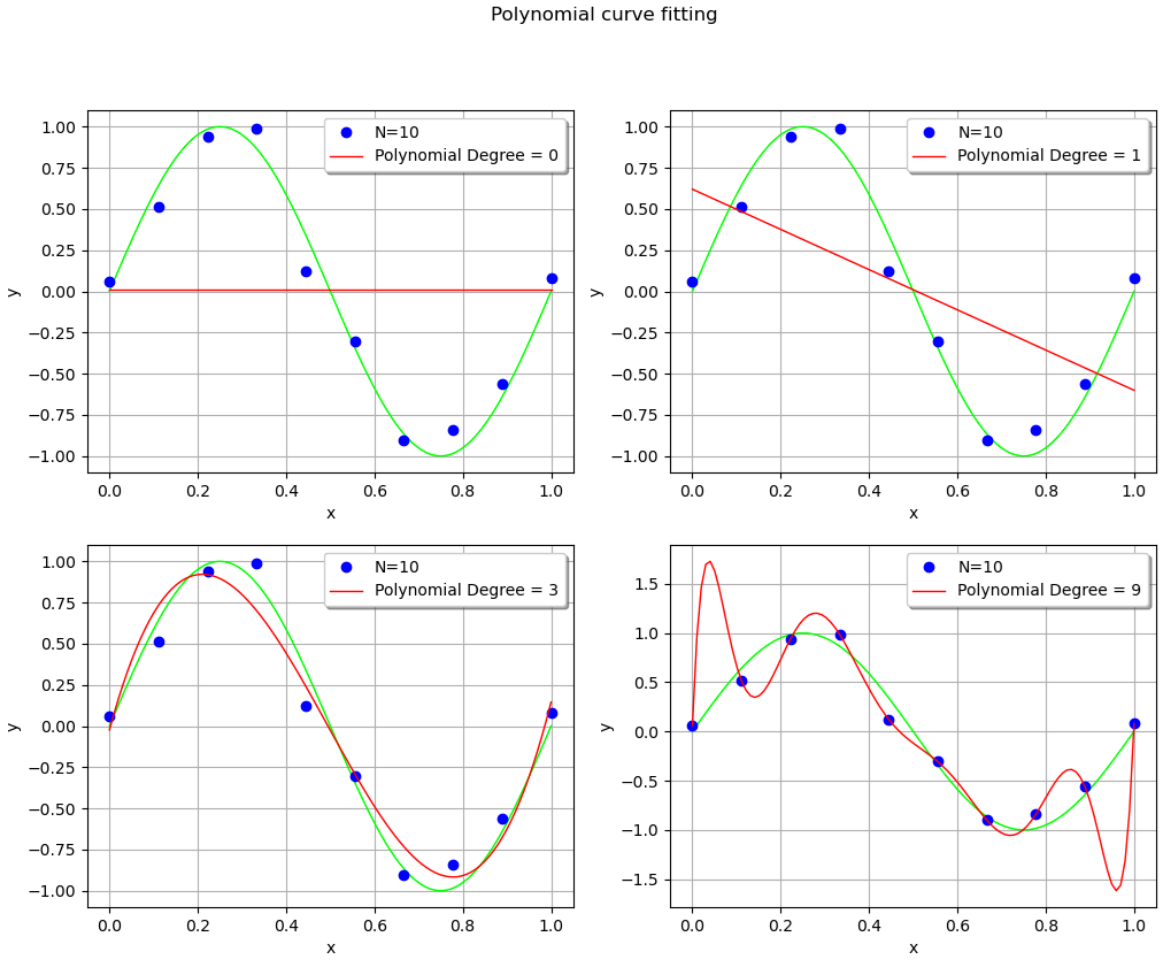
- Насколько хорошо модель обобщает данные, можно охарактеризовать по разнице между её эффективностью на данных, которые она видела, и на данных, которые она не видела.
- Если модель сделать более «сложной», она сможет выучить более «сложные закономерности», но при этом мы рискуем просто **запомнить** обучающие данные, вместо того чтобы действительно чему-то на них **научиться**.



Смещение/дисперсия

- Смещение:
  - Склонность к определённым прогнозам.
  - Насколько модель в среднем неверна, независимо от обучающих данных?
- Дисперсия:
  - Насколько чувствительна модель к изменениям в данных для обучения?
  - Небольшие изменения в наборе данных → значительные изменения в нашей модели и её прогнозах.
- Хорошая модель должна быть одновременно устойчивой и гибкой: способной фиксировать изменчивые и сложные данные, но при этом достаточно надёжной, чтобы делать обобщения, выходящие за рамки обучающих выборок.





Как различные настройки гиперпараметров («сложность модели») могут влиять на обобщение

Смещение/дисперсия

- При обучении (или настройке гиперпараметров) нам важно обобщение.
  - Нам необходимо сохранить небольшой фрагмент данных для тестирования модели в процессе обучения (валидационный набор).
  - Нам важно расхождение между метриками обучения и валидации, поскольку это хороший показатель итоговых возможностей обобщения модели на неизведанном тестовом наборе.



- Это подразумевает баланс между ошибками смещения и дисперсии.
  - Недообучение: модель плохо работает как на обучающих, так и на проверочных данных.
    - Это означает, что вы, вероятно, можете увеличить сложность модели без значительного ущерба для эффективности обобщения.
    - С точки зрения смещения/дисперсии это означает, что у вас высокая ошибка смещения.
  - Переобучение: модель отлично работает на обучающих данных, но плохо на проверочных.
    - Это означает, что наша модель в каком-то смысле слишком сложна и начала запоминать, а не обучаться; её необходимо масштабировать.
    - С точки зрения смещения/дисперсии это означает, что у вас высокая ошибка дисперсии.

Важные выводы

- Машинное обучение — это создание шаблонов
- Пайплайн машинного обучения
  - Определение проблемы
  - Подготовка данных
  - Определение модели и функции потерь
  - Обучение модели
  - Сообщение о результатах
- Часто существует компромисс между сложностью модели и её обобщением, известный как компромисс смещения-дисперсии.